

Linux Embebido – Módulo 4

Dispositivos de carácter: anatomía de un driver y GPIO real

Ing. Marín Ariel · Fabrizio Carlassara (BIDOC) ·

Pablo Gómez (BIDOC) · Alexis Hernández de la Cruz (BIS)

Técnicas Digitales III

Laboratorio de Sistemas Embebidos

Universidad Tecnológica Nacional

Facultad Regional Avellaneda

Departamento de Ingeniería Electrónica

Índice

1. Objetivos del módulo	3
2. Tipos de dispositivo en Linux	3
3. Anatomía de un driver de carácter	4
3.1. Major y minor number	4
3.2. <code>struct file_operations</code> : la tabla de despacho	4
3.3. <code>struct cdev</code> y el ciclo de vida del registro	5
3.4. Creación automática del nodo en <code>/dev</code>	5
4. La interfaz <code>open/release/read/write</code>	6
5. Describir el LED y el botón en el Device Tree	7
5.1. El overlay	7
5.2. Compilación y carga del overlay	8
6. GPIO desde el kernel: la API <code>gpiod</code>	9
6.1. Por qué no <code>/sys/class/gpio</code>	9
6.2. Obtener un GPIO desde un nodo del Device Tree	9
6.3. Leer y escribir el valor	10
7. Práctica: el dispositivo <code>/dev/ledbtn</code>	10
7.1. Montaje físico	11
7.2. El overlay: <code>ledbtn-overlay.dts</code>	11
7.3. El driver: <code>ledbtn.c</code>	11
7.4. Makefile	15
7.5. Compilación, carga y prueba	15
8. Cierre conceptual	16

Presentación del módulo

En el Módulo 2 se compiló y cargó un módulo de kernel “pelado”: código que corre en espacio de kernel pero que no expone ninguna interfaz a espacio de usuario más allá de los mensajes de `dmesg`. En este módulo se da el paso que falta para tener un **driver** en el sentido completo: un dispositivo de carácter con un nodo en `/dev` que cualquier aplicación puede abrir, leer y escribir con las llamadas al sistema estándar. El objetivo es doble. Primero, entender la anatomía interna de un driver de carácter: qué es un *major/minor number*, qué rol cumple `struct file_operations` como tabla de despacho, y cómo se registra un `cdev` para que el nodo aparezca solo en `/dev`. Segundo, controlar hardware real – un LED y un botón por GPIO – usando la API moderna `gpiod`, describiendo ese hardware en un overlay de Device Tree propio en lugar de escribir números de pin a mano en el código C. Al terminar el módulo, el alumno habrá escrito un driver completo que expone `/dev/ledbtn`, controlable con `echo` y `cat` desde la terminal.

1. Objetivos del módulo

Al finalizar este módulo, el alumno debería poder:

- Explicar la diferencia entre un módulo de kernel sin interfaz de usuario (Módulo 2) y un dispositivo de carácter con nodo en `/dev`.
- Describir la anatomía de un driver de carácter: *major/minor number*, `struct file_operations` y `struct cdev`.
- Registrar un dispositivo de carácter dinámicamente y lograr que su nodo en `/dev` se cree solo, sin `mknod` manual.
- Escribir un overlay de Device Tree que describa hardware propio (un LED y un botón) y localizarlo desde un módulo con `of_find_compatible_node`.
- Obtener y controlar GPIO real desde el kernel con la API `gpiod`: `fwnode_gpiod_get_index`, `gpiod_set_value`, `gpiod_get_value`, `gpiod_put`.
- Implementar `open/release/read/write` mínimos para exponer ese hardware a espacio de usuario.
- Probar el dispositivo completo con `echo` y `cat` sobre `/dev/ledbtn`.

2. Tipos de dispositivo en Linux

Linux clasifica los dispositivos que expone en `/dev` en tres grandes familias [2, cap. 1]:

- **Dispositivos de carácter:** transfieren datos como un flujo secuencial de bytes, sin estructura de bloques ni acceso aleatorio obligatorio. Se acceden con `open`, `read`, `write`, `close`,

igual que un archivo común. Ejemplos: terminales serie (`/dev/ttyAMA0`), `/dev/null`, `/dev/random`, y el dispositivo que se construye en este módulo.

- **Dispositivos de bloque:** transfieren datos en bloques de tamaño fijo y soportan acceso aleatorio, con un subsistema de caché e I/O propio. Ejemplo: `/dev/mmcblk0` (la tarjeta SD de la Raspberry Pi).
- **Dispositivos de red:** no tienen un nodo en `/dev`; se acceden a través de la API de sockets y se identifican por nombre de interfaz (`eth0`, `wlan0`).

Este módulo se concentra exclusivamente en dispositivos de carácter: son los más simples de escribir y los que mejor ilustran cómo el kernel conecta una llamada al sistema de espacio de usuario con código de un driver específico.

3. Anatomía de un driver de carácter

3.1. Major y minor number

Todo dispositivo de carácter se identifica con un par de números: el **major number** identifica el driver (todos los dispositivos que maneja ese driver comparten el mismo major), y el **minor number** identifica una instancia particular dentro de ese driver. El comando `ls -l /dev` los muestra en lugar del tamaño de archivo:

```
$ ls -l /dev/ttyAMA0
crw-rw---- 1 root dialout 204, 64 ago 9 10:00 /dev/ttyAMA0
```

La `c` inicial indica dispositivo de carácter; 204 es el major, 64 el minor. El archivo `/proc/devices` lista todos los majors registrados en el sistema en un momento dado.

Antiguamente los majors se asignaban a mano, con el riesgo de colisionar con otro driver. La práctica moderna es pedirle al kernel un major libre con `alloc_chrdev_region`, en lugar de fijar uno [2, cap. 3], [3].

3.2. struct file_operations: la tabla de despacho

Cuando una aplicación llama a `read(fd, ...)` sobre un archivo, el kernel no sabe de antemano si `fd` corresponde a un archivo en disco, una tubería o un dispositivo. La capa que resuelve esa ambigüedad es el **VFS** (*Virtual File System*): cada `struct file` abierto lleva un puntero a una `struct file_operations`, que es literalmente una tabla de punteros a función – una *vtable* – con una entrada por cada operación posible (`open`, `read`, `write`, `release`, ...) [4]. Un driver de carácter no reimplementa `read()`; completa esa tabla con sus propias funciones, y el VFS se encarga de invocarlas en el momento correcto.

```
static const struct file_operations ledbtn_fops = {
    .owner = THIS_MODULE,
    .open = ledbtn_open,
    .release = ledbtn_release,
```

```
.read = ledbtn_read,  
.write = ledbtn_write,  
};
```

Cualquier campo que se deje sin completar queda en NULL; el kernel lo trata como “operación no soportada” y la llamada al sistema correspondiente devuelve un error (por ejemplo, `-EINVAL`) en lugar de fallar de forma indefinida.

3.3. `struct cdev` y el ciclo de vida del registro

`struct cdev` es la estructura que representa un dispositivo de carácter dentro del kernel: vincula un rango de major/minor con la `struct file_operations` que lo implementa. Registrar un dispositivo de carácter completo requiere tres pasos, en este orden [3]:

1. `alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME)`: le pide al kernel que reserve dinámicamente un major libre y un minor a partir de 0, para 1 dispositivo.
2. `cdev_init(&cdev, &fops)` seguido de `cdev_add(&cdev, dev_num, 1)`: inicializa la estructura `cdev` con la tabla de operaciones y la da de alta ante el kernel. A partir de `cdev_add`, el dispositivo ya es alcanzable si algo abre ese major/minor – incluso antes de que exista un nodo en `/dev`.
3. En la descarga del módulo, los pasos se deshacen en orden inverso: `cdev_del` y luego `unregister_chrdev_region`.

3.4. Creación automática del nodo en `/dev`

`cdev_add` registra el dispositivo ante el kernel, pero no crea ningún archivo en `/dev`. Se podría crear a mano con `mknod`, pasándole el major y el minor exactos – fue así durante años –, pero es frágil: si el major cambia entre cargas (algo esperable al usar `alloc_chrdev_region`), el `mknod` manual queda desactualizado. La solución moderna es dejar que `udev` cree el nodo automáticamente, anunciando el dispositivo con `class_create` y `device_create` [5]:

```
ledbtn_class = class_create(CLASS_NAME);  
ledbtn_device = device_create(ledbtn_class, NULL, dev_num, NULL,  
    DEVICE_NAME);
```

`class_create` registra una “clase” de dispositivos en `/sys/class/` (visible luego en `/sys/class/td3/`); `device_create` anuncia una instancia concreta de esa clase con el major/minor ya asignado. Ese anuncio dispara un evento `uevent` que `udev`, corriendo en espacio de usuario, escucha para crear `/dev/ledbtn` automáticamente, con los permisos por defecto de esa clase de dispositivo.

Nota

La firma de `class_create` **cambió** en la versión 6.4 del kernel: hasta entonces recibía dos argumentos, `class_create(THIS_MODULE, nombre)`; desde 6.4 solo recibe el nombre, `class_create(nombre)`. Es una fuente común de errores de compilación al copiar ejemplos de libros o tutoriales viejos (incluido *Linux Device Drivers*, que usa la firma antigua): el kernel de la Pi en este curso es lo bastante reciente como para requerir la firma nueva. Verificar siempre la firma vigente en `Documentation/driver-api/driver-model/class.rst` [5] o con Elixir [10] antes de copiar un ejemplo de otra fuente.

La Figura 1 resume el camino completo, desde la llamada al sistema de espacio de usuario hasta el hardware.

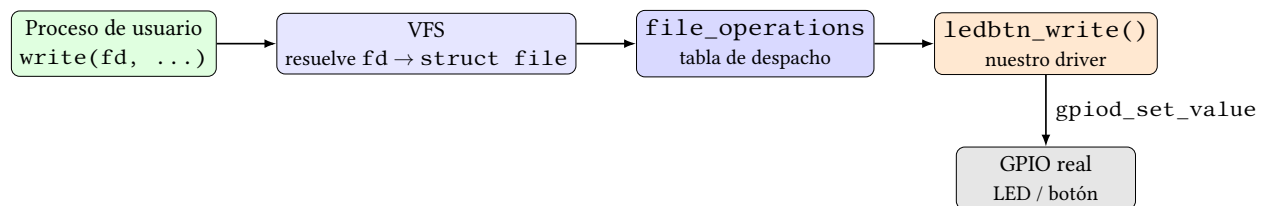


Figura 1: Camino de una llamada `write()` desde espacio de usuario hasta el hardware. El VFS no conoce a `ledbtn_write` directamente: lo encuentra a través del puntero `.write` de la tabla `file_operations` que el driver registró con `cdev_add`.

4. La interfaz open/release/read/write

Las cuatro operaciones básicas de un dispositivo de carácter, con la firma que exige `struct file_operations` [4]:

- `int open(struct inode *inode, struct file *file)`: se ejecuta cuando algo abre el nodo. Es el lugar típico para reservar recursos por instancia (buffers, contar referencias). El driver de este módulo no necesita estado por apertura, así que devuelve 0 sin hacer nada más.
- `int release(struct inode *inode, struct file *file)`: el contrapunto de `open`, se ejecuta al cerrar. Misma situación: no hay nada que liberar acá.
- `ssize_t read(struct file *file, char __user *buf, size_t len, loff_t *off)`: copia datos *del driver hacia* `buf`, un puntero de **espacio de usuario**. Devuelve la cantidad de bytes efectivamente leídos, o 0 para indicar fin de archivo.
- `ssize_t write(struct file *file, const char __user *buf, size_t len, loff_t *off)`: el sentido inverso, copia datos de `buf` hacia el driver.

El calificador `__user` en `buf` no es cosmético: marca un puntero que vive en el espacio de direcciones del proceso que hizo la llamada, no en el del kernel. El kernel **no puede desreferenciarlo directamente** – ese puntero podría apuntar a memoria que no le pertenece o no está mapeada en este momento –, por eso existen las funciones `copy_to_user` y `copy_from_user`: validan el rango y hacen la copia de forma segura entre los dos espacios de direcciones [2, cap. 3], [4].

Nota

Este módulo usa `read/write` con un único byte, lo mínimo necesario para que la práctica del LED y el botón funcione. El manejo completo de `off` (posición dentro del archivo, relevante para lecturas parciales o `lseek`), de buffers de tamaño arbitrario, y de `ioctl` para configurar el dispositivo desde espacio de usuario se cubre en profundidad recién en el Módulo 5.

5. Describir el LED y el botón en el Device Tree

En el Módulo 2 se leyeron propiedades de nodos *ya existentes* en el Device Tree base de la Pi. Acá se hace lo inverso: describir hardware propio – un LED y un botón que no están en ningún DTB de fábrica – agregando un nodo nuevo con un **overlay**, y localizarlo desde el driver con la misma función `of_find_compatible_node` que ya se usó en la Práctica 4 del Módulo 2.

5.1. El overlay

```
/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2711";

    fragment@0 {
        target-path = "/";
        __overlay__ {
            ledbtn: ledbtn {
                compatible = "td3,ledbtn";
                led-gpios = <&gpio 17 0>;
                boton-gpios = <&gpio 27 1>;
                status = "okay";
            };
        };
    };
};
```

Elementos nuevos respecto a la sintaxis DTS ya vista en el Módulo 2:

- `/plugin/;`: marca este archivo como un overlay (un fragmento que se aplica sobre un

árbol base ya cargado), no un Device Tree completo.

- `fragment@0 { target-path = "/"; __overlay__ { ... } }`: todo lo que está dentro de `__overlay__` se inserta en el árbol base en el punto que indica `target-path` – acá, la raíz. Un overlay puede tener varios `fragment@N` para tocar distintos puntos del árbol a la vez.
- `compatible = "td3,ledbtn"`: un string inventado para este curso. No corresponde a ningún driver del kernel mainline, así que el nodo queda sin *binding* automático: nuestro módulo lo va a encontrar a mano con `of_find_compatible_node`. Vincular automáticamente un driver a este tipo de nodo (con `probe()` y `platform_driver`) es exactamente lo que se cubre en el Módulo 6.
- `led-gpios / boton-gpios`: la convención de Device Tree para declarar una referencia a GPIO es que la propiedad termine en `-gpios` [8]. El valor es un *gpio-specifier*: el phandle del controlador GPIO (`&gpio`, la etiqueta del controlador en el árbol base de la Pi), el número de pin BCM, y una celda de *flags*.
- La celda de flags distingue polaridad: 0 es activo en alto, 1 es activo en bajo. El LED se cablea activo en alto (0): al pin en 1 el LED enciende. El botón se cablea activo en bajo (1, con un pull-up externo a 3.3 V): al presionarlo, el pin baja a 0 V. Gracias a la flag, `gpiod_get_value` devuelve 1 cuando el botón está presionado **sin importar la polaridad física del cableado** – ese es justamente el motivo de que la celda de flags exista: desacopla la lógica del driver de cómo quedó armado el circuito.

5.2. Compilación y carga del overlay

```
# Compilar el overlay a formato binario (.dtbo)
dtc -@ -I dts -O dtb -o ledbtn.dtbo ledbtn-overlay.dts

# Copiarlo al directorio de overlays de la Pi
sudo cp ledbtn.dtbo /boot/firmware/overlays/

# Activarlo agregando esta linea a /boot/firmware/config.txt
echo "dtoverlay=ledbtn" | sudo tee -a /boot/firmware/config.txt

# Reiniciar para que el bootloader lo aplique
sudo reboot
```

El flag `-@` le indica a `dtc` que genere información de símbolos en el `.dtbo`, necesaria para que el mecanismo de overlays de la Pi pueda resolver la referencia `&gpio` contra el árbol base en tiempo de arranque [9].

Nota

Reiniciar para probar cada cambio de overlay es lento. La Raspberry Pi también permite aplicar y quitar overlays en caliente con la herramienta `dtoverlay`, sin editar `config.txt` ni reiniciar – útil mientras se está iterando sobre el propio overlay: `sudo dtoverlay ledbtn` para aplicarlo, `dtoverlay -l` para listar los aplicados, `sudo dtoverlay -r ledbtn` para quitarlo [9].

Para verificar que el nodo quedó incorporado al árbol en vivo, se puede reusar exactamente la misma técnica del Módulo 2 de navegar `/sys/firmware/devicetree/base/`:

```
ls /sys/firmware/devicetree/base/ledbtn/
cat /sys/firmware/devicetree/base/ledbtn/compatible
```

6. GPIO desde el kernel: la API `gpiod`

6.1. Por qué no `/sys/class/gpio`

Versiones viejas de Linux exponían los GPIO directamente a espacio de usuario a través de `/sys/class/gpio` (`echo 17 >export`, después leer/escribir `/sys/class/gpio/gpio17/value`). Esa interfaz está **deprecada** desde el kernel 4.8 y puede no estar disponible en instalaciones recientes de Raspberry Pi OS [7]. El reemplazo para espacio de usuario es la interfaz de dispositivo de carácter `/dev/gpiochipN` (biblioteca `libgpiod`), pero eso no es lo que se necesita acá: este driver corre en **espacio de kernel** y controla el GPIO directamente con la API de consumidor `gpiod_*`, declarada en `<linux/gpio/consumer.h>` [6].

6.2. Obtener un GPIO desde un nodo del Device Tree

Cuando un driver tiene un `struct device` propio (típicamente entregado por `probe()` en un `platform_driver`, como se va a ver en el Módulo 6), `gpiod_get(dev, con_id, flags)` alcanza. Acá no hay `struct device`: el nodo se encontró a mano con `of_find_compatible_node`. Para ese caso, la API de consumidor ofrece `fwnode_gpiod_get_index`, que trabaja sobre un `struct fwnode_handle *` en lugar de un `struct device *` [6]:

```
struct gpio_desc *fwnode_gpiod_get_index(struct fwnode_handle *fwnode,
                                         const char *con_id, int index,
                                         enum gpiod_flags flags,
                                         const char *label);
```

`of_fwnode_handle(node)` convierte el `struct device_node *` que ya se tiene en el `struct fwnode_handle *` que pide esta función. A diferencia de `gpiod_get_from_of_node` (ver nota), acá `con_id` es el mismo tipo de identificador **corto** que usa `gpiod_get`: la función le agrega sola el sufijo `-gpios`, así que alcanza con

"led"/"boton", no "led-gpios". `flags` fija la dirección inicial: `GPIO_OUT_LOW` (salida, arranca en 0) para el LED, `GPIO_IN` para el botón.

Nota

La documentación del kernel también describe `gpiod_get_from_of_node` – misma idea, pero recibe directamente un `struct device_node *` y el nombre **completo** de la propiedad ("led-gpios"). En el kernel de la Raspberry Pi que usa este curso esa función no resolvió el GPIO correctamente; `fwnode_gpiod_get_index` sí, y es la que efectivamente usa el driver de la Práctica. Vale la pena tenerlo presente: dos funciones documentadas para el mismo caso de uso no siempre se comportan igual en la versión de kernel que se tiene enfrente – ante la duda, hay que confiar en lo que compiló y funcionó en la Pi real, no solo en lo que dice la documentación.

6.3. Leer y escribir el valor

Una vez obtenido el `struct gpio_desc *`, las operaciones básicas son [6]:

- `void gpiod_set_value(struct gpio_desc *desc, int value)`: fija el valor lógico de una salida (0/1), ya corregido por la polaridad que se definió en el `gpio-specifier` del Device Tree.
- `int gpiod_get_value(struct gpio_desc *desc)`: lee el valor lógico de una entrada, también corregido por polaridad.
- `void gpiod_put(struct gpio_desc *desc)`: libera la referencia al GPIO. Imprescindible en la descarga del módulo, igual que `of_node_put` para nodos del Device Tree.

Nota

Existen las variantes `gpiod_set_value_cansleep` / `gpiod_get_value_cansleep` para GPIO expuestos por expansores conectados a un bus lento como I2C o SPI, donde leer/escribir implica una transacción que puede bloquear. Los GPIO nativos del SoC de la Pi, como los de esta práctica, nunca bloquean y usan las variantes sin sufijo. Usar las variantes `cansleep` en contexto atómico (por ejemplo, dentro de un manejador de interrupciones) es un error de programación que el kernel detecta y reporta en `dmesg` [6].

La Figura 2 conecta todo lo visto en esta sección: desde el archivo `.dts` hasta el pin físico que el driver termina leyendo o escribiendo.

7. Práctica: el dispositivo `/dev/ledbtn`

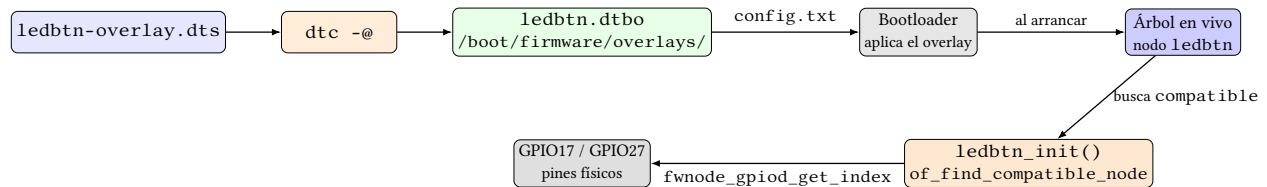


Figura 2: Desde el archivo fuente del overlay hasta el pin físico. El bootloader mezcla el .dtbo con el árbol base antes de pasarle el resultado al kernel; el módulo ubica el nodo **ledbtn** a mano en **module_init** y desde ahí obtiene los descriptores de GPIO reales.

7.1. Montaje físico

- **LED:** ánodo a GPIO17 (pin físico 11) a través de una resistencia de 220 Ω ; cátodo a GND (por ejemplo, pin físico 9).
- **Botón:** una pata a GPIO27 (pin físico 13); la otra a GND. Resistencia pull-up externa de 10 k Ω entre GPIO27 y 3.3 V (pin físico 1), para que el pin quede en alto cuando el botón no está presionado y baje a 0 V al presionarlo – la polaridad “activo en bajo” que ya se declaró en la flag del overlay.

7.2. El overlay: **ledbtn-overlay.dts**

El archivo completo es el que se armó en la Sección 5.

7.3. El driver: **ledbtn.c**

```

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/gpio/consumer.h>
#include <linux/uaccess.h>
#include <linux/err.h>

#define DEVICE_NAME "ledbtn"
#define CLASS_NAME "td3"

MODULE_LICENSE("GPL");
MODULE_AUTHOR("TD3 -- UTN FRA");
MODULE_DESCRIPTION("Char device de ejemplo: LED + boton por GPIO real");

static dev_t dev_num;
static struct cdev ledbtn_cdev;
static struct class *ledbtn_class;
static struct device *ledbtn_device;
  
```

```
static struct gpio_desc *led_gpio;
static struct gpio_desc *boton_gpio;

static int ledbtn_open(struct inode *inode, struct file *file)
{
    return 0;
}

static int ledbtn_release(struct inode *inode, struct file *file)
{
    return 0;
}

static ssize_t ledbtn_read(struct file *file, char __user *buf,
                           size_t len, loff_t *off)
{
    char estado;

    if (*off > 0)
        return 0; /* ya se entrego un byte: fin de archivo */

    estado = gpiod_get_value(boton_gpio) ? '1' : '0';

    if (copy_to_user(buf, &estado, 1))
        return -EFAULT;

    *off += 1;
    return 1;
}

static ssize_t ledbtn_write(struct file *file, const char __user *buf,
                            size_t len, loff_t *off)
{
    char valor;

    if (len < 1)
        return -EINVAL;

    if (copy_from_user(&valor, buf, 1))
        return -EFAULT;

    gpiod_set_value(led_gpio, valor == '1' ? 1 : 0);

    /* Se descarta el resto del buffer (p.ej. el '\n' que agrega "echo")
     */
    return len;
}
```

```

static const struct file_operations ledbtn_fops = {
    .owner = THIS_MODULE,
    .open = ledbtn_open,
    .release = ledbtn_release,
    .read = ledbtn_read,
    .write = ledbtn_write,
};

static int __init ledbtn_init(void)
{
    struct device_node *node;
    int ret;

    node = of_find_compatible_node(NULL, NULL, "td3,ledbtn");
    if (!node) {
        printk(KERN_ERR "ledbtn: no se encontro el nodo 'td3,ledbtn' en el
            Device Tree\n");
        return -ENODEV;
    }

    led_gpio = fwnode_gpiod_get_index(of_fwnode_handle(node), "led", 0,
        GPIOD_OUT_LOW, "led");
    if (IS_ERR(led_gpio)) {
        printk(KERN_ERR "ledbtn: no se pudo obtener el GPIO del LED\n");
        ret = PTR_ERR(led_gpio);
        goto err_node;
    }

    boton_gpio = fwnode_gpiod_get_index(of_fwnode_handle(node), "boton",
        0,
        GPIOD_IN, "boton");
    if (IS_ERR(boton_gpio)) {
        printk(KERN_ERR "ledbtn: no se pudo obtener el GPIO del boton\n");
        ret = PTR_ERR(boton_gpio);
        goto err_led;
    }

    of_node_put(node);

    ret = alloc_chrdev_region(&dev_num, 0, 1, DEVICE_NAME);
    if (ret < 0) {
        printk(KERN_ERR "ledbtn: no se pudo reservar major/minor\n");
        goto err_boton;
    }

    cdev_init(&ledbtn_cdev, &ledbtn_fops);
    ledbtn_cdev.owner = THIS_MODULE;
    ret = cdev_add(&ledbtn_cdev, dev_num, 1);
}

```

```

if (ret < 0) {
    printk(KERN_ERR "ledbtn: no se pudo registrar el cdev\n");
    goto err_chrdev;
}

ledbtn_class = class_create(CLASS_NAME);
if (IS_ERR(ledbtn_class)) {
    printk(KERN_ERR "ledbtn: no se pudo crear la clase\n");
    ret = PTR_ERR(ledbtn_class);
    goto err_cdev;
}

ledbtn_device = device_create(ledbtn_class, NULL, dev_num, NULL,
    DEVICE_NAME);
if (IS_ERR(ledbtn_device)) {
    printk(KERN_ERR "ledbtn: no se pudo crear el dispositivo\n");
    ret = PTR_ERR(ledbtn_device);
    goto err_class;
}

printk(KERN_INFO "ledbtn: modulo cargado, /dev/%s listo\n",
    DEVICE_NAME);
return 0;

err_class:
    class_destroy(ledbtn_class);
err_cdev:
    cdev_del(&ledbtn_cdev);
err_chrdev:
    unregister_chrdev_region(dev_num, 1);
err_boton:
    gpiod_put(boton_gpio);
err_led:
    gpiod_put(led_gpio);
    return ret;
err_node:
    of_node_put(node);
    return ret;
}

static void __exit ledbtn_exit(void)
{
    device_destroy(ledbtn_class, dev_num);
    class_destroy(ledbtn_class);
    cdev_del(&ledbtn_cdev);
    unregister_chrdev_region(dev_num, 1);
    gpiod_put(boton_gpio);
    gpiod_put(led_gpio);
}

```

```
    printk(KERN_INFO "ledbtn: modulo descargado\n");  
}  
  
module_init(ledbtn_init);  
module_exit(ledbtn_exit);
```

Nota

`ledbtn_write` ignora cualquier byte después del primero y devuelve `len` completo, no solo 1. Es deliberado: `echo "1" >/dev/ledbtn` en realidad escribe dos bytes ('1' y el '\n' final). Si `write` devolviera 1, `echo` interpretaría que quedó un byte sin escribir y volvería a llamar a `write` con el '\n' solo, generando una segunda invocación innecesaria. Devolver `len` le dice al VFS “se consumió todo lo que me pasaste”, que es la convención esperada para un dispositivo que no necesita cada byte individual del buffer.

7.4. Makefile

```
obj-m += ledbtn.o  
  
KDIR := /lib/modules/$(shell uname -r)/build  
  
all:  
    $(MAKE) -C $(KDIR) M=$(PWD) modules  
  
clean:  
    $(MAKE) -C $(KDIR) M=$(PWD) clean
```

7.5. Compilación, carga y prueba

Con el overlay ya aplicado (Sección 5):

```
# Compilar el modulo  
make  
  
# Cargarlo  
sudo insmod ledbtn.ko  
  
# Verificar que el nodo se creo solo, sin mknod manual  
ls -l /dev/ledbtn  
  
# Prender el LED  
echo "1" | sudo tee /dev/ledbtn  
  
# Apagar el LED  
echo "0" | sudo tee /dev/ledbtn
```

```
# Leer el estado del boton (probar sin presionar y presionado)
cat /dev/ledbtn

# Ver los mensajes del modulo
dmesg | tail -5

# Descargar el modulo
sudo rmmmod ledbtn
```

Salida esperada de `ls -l /dev/ledbtn` (el `major` concreto depende de qué otros `major`s dinámicos estén ocupados en ese arranque, por eso se pidió con `alloc_chrdev_region` en lugar de fijarlo):

```
crw-rw---- 1 root td3 238, 0 ago 9 10:00 /dev/ledbtn
```

8. Cierre conceptual

Este módulo cerró la distancia entre “módulo de kernel que solo escribe en `dmesg`” (Módulo 2) y “driver real que una aplicación puede usar”. Los puntos clave para retener:

- Un dispositivo de carácter se identifica por *major/minor*, y el VFS despacha cada llamada al sistema a través de la tabla `struct file_operations` que el driver registró con `cdev_add`.
- `class_create` + `device_create` delegan en `udev` la creación del nodo en `/dev`: no hace falta `mknod` manual, y por eso importa revisar la firma vigente de estas funciones en la documentación del kernel en uso, no en un ejemplo viejo.
- Describir hardware propio en un **overlay** de Device Tree (en lugar de escribir números de pin en el código C) reutiliza exactamente la misma técnica de lectura del Módulo 2 (`of_find_compatible_node`), aplicada ahora a un nodo que el propio curso definió.
- La API `gpiod` reemplaza tanto al `/sys/class/gpio` deprecado como a la vieja API entera de `gpio_request/gpio_set_value`, y separa la lógica del driver (“prender”, “leer”) de la polaridad física del cableado gracias a la celda de flags del `gpio-specifier`.

Lo que queda pendiente, a propósito: `read/write` se usaron con el mínimo indispensable para que la práctica funcionara. El Módulo 5 profundiza esa interfaz – manejo correcto de `off`, buffers de tamaño arbitrario, y `ioctl` para configurar el dispositivo desde espacio de usuario – sobre este mismo driver. El Módulo 6, a su vez, reemplaza la búsqueda manual de `of_find_compatible_node` en `module_init` por el modelo real de `platform_driver` con `probe()`, aplicado a un periférico existente (el UART) en lugar de uno inventado para el curso.

Referencias

- [1] Vasquez, F. y Simmonds, C. (2021). *Mastering Embedded Linux Programming*, 3.^a edición. Packt Publishing.
- Cap. 11, *Interfacing with Device Drivers* – panorama de las distintas formas en que Linux expone hardware a espacio de usuario, incluyendo dispositivos de carácter y GPIO.
- [2] Corbet, J., Rubini, A. y Kroah-Hartman, G. (2005). *Linux Device Drivers*, 3.^a edición. O'Reilly Media. Disponible libremente en <https://lwn.net/Kernel/LDD3/>.
- Cap. 1, *An Introduction to Device Drivers* – panorama de las familias de dispositivos (carácter, bloque, red).
 - Cap. 3, *Char Drivers* – anatomía completa de un driver de carácter: major/ minor number, file_operations, registro, copy_to_user/ copy_from_user. La API de registro que usa el libro es la de una versión vieja del kernel; la vigente está en Documentation/driver-api/ [3].
- [3] The Linux Kernel development community. *The Linux Kernel API* – sección de dispositivos de carácter. <https://docs.kernel.org/core-api/kernel-api.html>. Referencia generada directamente de los comentarios *kernel-doc* del código fuente: firmas vigentes de alloc_chrdev_region, cdev_init, cdev_add, cdev_del y unregister_chrdev_region.
- [4] The Linux Kernel development community. *Overview of the Linux Virtual File System*. <https://docs.kernel.org/filesystems/vfs.html>. Describe struct file_operations campo por campo, y el contrato de read/write con los punteros de espacio de usuario.
- [5] The Linux Kernel development community. *Class Basics* y documentación asociada del modelo de dispositivos. <https://docs.kernel.org/driver-api/driver-model/class.html>. Referencia de class_create/class_destroy/device_create: útil en particular para verificar la firma vigente de class_create, que cambió en la versión 6.4 del kernel.
- [6] The Linux Kernel development community. *GPIO Descriptor Consumer Interface*. <https://docs.kernel.org/driver-api/gpio/consumer.html>. Referencia completa de la API gpiod_*: obtención de descriptors (incluyendo fwnode_gpiod_get_index y gpiod_get_from_of_node), lectura/escritura de valor, y las variantes cansleep para GPIO detrás de buses lentos.
- [7] The Linux Kernel development community. *Sysfs Interface for GPIO* (deprecated ABI). <https://docs.kernel.org/admin-guide/gpio/sysfs.html>. Do-

cumenta – y marca como deprecada – la interfaz `/sys/class/gpio` para controlar GPIO desde espacio de usuario sin escribir un driver.

- [8] The Linux Kernel development community. *Documentation/devicetree/bindings/gpio/gpio.txt*. <https://elixir.bootlin.com/linux/latest/source/Documentation/devicetree/bindings/gpio/gpio.txt>. Especificación del binding genérico de GPIO en Device Tree: la convención de nombrar las propiedades `<nombre>-gpios` y el formato del gpio-specifier (phandle, pin, flags).
- [9] Raspberry Pi Ltd. *Device Trees, overlays, and parameters* – Raspberry Pi Documentation. <https://www.raspberrypi.com/documentation/computers/configuration.html>. Mecánica específica de la Raspberry Pi para compilar, instalar y aplicar overlays: el directorio `/boot/firmware/overlays/`, la directiva `dtoverlay=` en `config.txt`, y la herramienta `dtoverlay` para aplicar overlays en caliente.
- [10] Bootlin. *Elixir Cross Referencer – Linux source code*. <https://elixir.bootlin.com/linux/latest/source>. Herramienta de navegación del código fuente del kernel de Linux con referencias cruzadas: permite verificar la firma exacta de cualquier función (por ejemplo, `class_create` en la versión de kernel que corresponda) directamente en el código fuente.